
Ablating Real Proof Developments: A Compile-Validated Proof-Repair Benchmark over 57 Lean 4 Repositories

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Benchmarks for machine proof synthesis are dominated by competition mathemat-
2 ics, where a problem is a self-contained statement with no surrounding development.
3 Real proof engineering is not that: it is repair inside an existing library, where a
4 lemma must be re-derived so that everything downstream of it still goes through.
5 We construct such a task by syntactic ablation of real repositories — pick a theorem,
6 take its in-file transitive dependency closure, delete one lemma from that closure,
7 and hole the proofs that used it — and we validate every problem by running the
8 compiler on *both* sides: the challenge must compile with its holes, and the recorded
9 ground truth must compile hole-free, each against a pinned `.olean` closure. The
10 result is 43,410 challenges mined from 57 public Lean 4 repositories under two hol-
11 ing strategies over matched selections. Evaluating three models on a 113-problem
12 matched sample at a 50-turn budget separates the strongest from the rest — 49.1%,
13 29.2% and 30.8% macro PASS under leaf holing — but the more consequential
14 result is what the second half of the oracle catches. The strongest model in the grid
15 never fails honestly: of 103 scorable problems it passes 51 and is caught tampering
16 on 50, with zero turn-limits, zero give-ups and zero non-compiling submissions. A
17 scorer that checked only “compiles, no holes” would have reported 98.1% rather
18 than 49.5%. Tamper rate ranks with capability across the three models (48.5% for
19 the strongest model against 26.2% and 26.8% for the other two), and a genuine
20 15/30/50/100-turn budget curve shows it does not fall when the turn budget is cut —
21 the strongest model tampers *most* at the lowest budget. The two holing strategies,
22 by contrast, do not differ in pass rate at this budget (McNemar $p = 0.75$ and 1.00
23 for the two general-purpose models); what they change is the composition of the
24 failures and the turn cost. Contamination is measured rather than asserted: 2 of
25 57 repositories were crawled by Software Heritage before The Stack v2 snapshot,
26 all 21,692 deleted lemmas are dated to their introducing commit, and a temporal
27 holdout finds no systematic pre-cutoff pass advantage.

1 Introduction

28 The public benchmarks that drive work on machine proof synthesis are overwhelmingly competition
29 mathematics: miniF2F, ProofNet, PutnamBench. A problem in those benchmarks is a formal
30 statement standing on its own, formalized from a natural-language competition problem, to be
31 proved from a general-purpose library. That is a real task, and progress on it is real progress. It
32 is not, however, the task that proof engineers spend their time on. A proof engineer works inside
33 a development that already exists, where lemmas have downstream users, where a change to one
34 statement propagates, and where “done” means the build is green.
35

We construct a benchmark for that second task by *ablating* real developments. Given a theorem in a real repository — we call it the *corollary* — we compute its transitive in-file dependency closure, delete one lemma from that closure, and replace with holes the proof steps that cited it. A solver is handed the resulting file and must produce a version that compiles with no holes. The deleted lemma’s original text is the recorded ground truth, but it is not the only acceptable answer: the oracle is the kernel, not string equality.

The discipline that makes this trustworthy, and which we regard as the main methodological contribution, is **two-sided compile validation**. Every shipped problem has been compiled twice before release: once as the challenge, to confirm it elaborates with its holes in place, and once as the ground truth, to confirm the recorded solution actually closes them. Extraction pipelines that skip the second check can ship unwinnable problems indefinitely without anyone noticing, because the scorer is never run on the gold answer. In our own pipeline the check is load-bearing in a way we did not anticipate: it is only as trustworthy as the build environment behind it, and three environment bugs cost us roughly 3,700 valid challenges before they were found — one repository went from 459 to 3,340 validated challenges with no change to the ablation logic at all.

Contributions.

1. **A compile-validated proof-repair corpus at scale** (§3, §4): 43,410 challenges from 57 real Lean 4 repositories, every one of them with challenge and ground truth compiled against pinned `.olean` closures.
2. **Reward hacking under a verified reward, measured** (§5.3): the rate at which a solver makes the file compile by deleting or weakening the theorem it was asked to re-prove. On our grid this rate ranks with model capability and reaches roughly half of all scorable attempts for the strongest model, which makes the statement-preservation guard the difference between a 98% result and a 50% result.
3. **Two holing strategies, measured rather than assumed** (§5.4): leaf-step holing and whole-body holing, compared on a matched problem set. The premise that the second is harder is *not* borne out at a 50-turn budget for any of the three models; what changes is the failure composition and the turn cost.
4. **A contamination analysis that is measured** (§5.5): a direct corpus-membership check against the dated Software Heritage snapshot underlying The Stack v2, a temporal hold-out over lemma introduction dates, a deletion-count sweep, and a verbatim recall probe, organised around the instance-level versus knowledge-level memorization distinction.
5. **A calibrated difficulty model** (§5.6): held-out discrimination *and* calibration, with the Murphy decomposition and a reliability diagram, not an AUC alone.

2 Related work

Competition-mathematics benchmarks. miniF2F [16], ProofNet [1], PutnamBench [12] and FIMO [9] formalize competition and textbook problems as standalone theorem statements. The statements are self-contained by construction: there is no ambient development, no downstream user of the theorem, and no build to keep green. This makes them clean instruments for measuring proving ability and poor instruments for measuring the ability to work inside an existing library. Our problems arrive with the library attached, and success is defined by what the surrounding file requires, not by the target statement alone.

Repository-scale extraction. LeanDojo [14] and LeanStep [5] extract training and evaluation data from Mathlib at repository scale, and CoqGym [13] does the analogous thing for Rocq. These do operate on real developments, which is the property we want. Two things differ. First, the extracted unit is typically a proof *state* paired with the next tactic, so the task is next-step prediction rather than re-deriving a whole lemma and repairing its users. Second, and more importantly for our purposes, the extracted artifacts are generally not validated as standalone compilable objects on both sides: the environment can replay a proof, but the released problem is not itself a file that has been shown to compile with holes and to compile hole-free once solved. That two-sided guarantee is what we add.

86 **Repair benchmarks with execution-based scoring.** SWE-bench [7] and its descendants have the
 87 task shape we want — repair inside a real repository, scored by execution — and they established
 88 that this shape is both tractable and discriminative. The oracle differs in kind. A test suite admits
 89 false positives: a patch can satisfy the tests and be wrong. A proof kernel does not, which is why
 90 a memorized proof that compiles is simply a correct proof, and why reward hacking in our setting
 91 takes the specific form of attacking the *statement* rather than the proof (§5.3).

92 **Contamination methodology.** Our §5.5 is assembled from established instruments rather than
 93 invented ones: temporal holdout, as in LiveCodeBench [6] and GSM1K [15]; perturbation robustness,
 94 as in GSM-Symbolic [11]; verbatim-recall probing under guided prompting [4]; and the finding that
 95 memorization scales with duplication count in the training corpus [2], which is why we do not use
 96 repository popularity as a contamination proxy. We add one instrument that the proof-benchmark
 97 literature has not used: because The Stack v2 [10] is constructed from a Software Heritage [3] graph
 98 snapshot with a published date, and Software Heritage records a dated crawl history per repository,
 99 corpus membership for a specific source repository can be checked directly rather than inferred
 100 (§5.5).

101 **The differentiator.** In one sentence: *real-repository proofs, posed as a repair task, with the*
 102 *compiler run on both the challenge and the ground truth before either ships.* Competition benchmarks
 103 have the compiler but not the repository; extraction corpora have the repository but not the two-sided
 104 validation; software-repair benchmarks have both but not a sound oracle.

105 3 Method

106 3.1 Ablation as proof repair

107 Fix a source file F from a repository pinned at a specific revision. For each theorem $c \in F$ that has a
 108 non-empty in-file dependency closure — we call c the *corollary* — let $\text{cl}(c)$ be the transitive closure
 109 of the lemmas in F that c ’s proof depends on. We draw a lemma $\ell \in \text{cl}(c)$, delete ℓ ’s declaration
 110 entirely, and replace with holes (sorry) the proof material in F that cited ℓ . The *challenge* is the
 111 resulting file; the *solution* is F with ℓ restored. Files whose theorems cite only out-of-file lemmas
 112 produce nothing, since there is nothing in-file to delete.

113 The solver’s task is to emit a version of the challenge file that compiles with no remaining holes. It is
 114 not asked to reproduce ℓ verbatim, and typically cannot: any re-derivation that discharges the holes is
 115 accepted. Scoring is by compilation against the repository’s prebuilt .olean closure at the pinned
 116 revision, plus a hole check and the statement-preservation guard of §3.7.

117 3.2 Two holing strategies

118 The two strategies differ in exactly one respect: how much of each *user* of ℓ survives.

- 119 • **Leaf holing** (corollary-leaves). Only the leaf tactic steps that cited ℓ are replaced. The
 120 rest of each user’s proof survives, so the solver sees a proof skeleton with gaps in it and can
 121 read the surrounding structure as a constraint on what the missing lemma must have said.
- 122 • **Whole-body holing** (corollary-whole). Each user’s entire proof body is replaced. The
 123 solver sees the statements and nothing else, and must reconstruct both ℓ and every proof that
 124 used it from the statements alone.

125 Whole-body holing is intended to be strictly harder, and the splits are published as *easy* and *hard* on
 126 that premise. It is a hypothesis, not a fact, and §5.4 tests it: at a 50-turn budget the pass rates do not
 127 separate, and what the strategies actually change is how a solver fails and how long it takes. Because
 128 both strategies are applied to the same deletion under the same seed, the comparison can be paired
 129 (§3.4).

130 3.3 Corollary-anchored slicing

131 Both strategies slice F down to a minimal dependency closure so that the emitted context stays
 132 small. The slice is anchored on the *corollary*, not on the holes. This is a subtle point with a large

consequence. The holes are the in-file users of ℓ — a property of ℓ , not of the corollary that motivated selecting it. Anchoring the slice on the holes makes the corollary invisible in the output, so two distinct corollaries whose closures both contain ℓ emit a *byte-identical* challenge under two different identifiers: the same problem shipped twice. In an earlier release this duplicated roughly half the mined corpus. Anchoring on the corollary makes each record the question it claims to be — *delete ℓ , then rebuild it so that **this** corollary still goes through* — with context cut to what that corollary actually depends on. Records are additionally deduplicated on the (challenge, solution) text, which is what a solver actually sees.

141 3.4 Matched selections and the challenge identifier

142 Each record carries a stable identifier

challenge_id = sha1₁₆(file path | seed | variant | deleted lemma names | holed theorem names),

143 truncated to 16 hex characters. The key deliberately omits the holing strategy, so the same deletion under leaf and whole-body holing carries the *same* identifier. This is what makes the two published splits a matched pair rather than two independent samples, and it is why §5.4 can use a paired test. It also joins outcomes to features exactly, which the difficulty model of §5.6 depends on.

147 3.5 Two-sided compile validation

148 Every mined record is run through a pre-flight check that splices it into a work copy of the built repository and compiles it. A record is kept only if:

- 150 1. the **challenge** compiles with its holes in place — it elaborates, so the solver is looking at a well-formed file and any error it sees is one it introduced; and
- 152 2. the **ground truth** compiles *hole-free* and axiom-free — the problem is winnable, and we have exhibited one winning answer.

154 Records failing (1) are recorded as malformed and dropped; records failing (2) — the recorded solution already contained a sorry or an axiom, so the problem is unwinnable — are recorded as sol_BAD and dropped. Both counts are published per repository alongside the kept records.

157 **Validation is a build problem.** A challenge counts as valid only if it really compiles, so this check is only as trustworthy as the environment behind it, and *a broken environment reports every challenge as malformed, which reads as bad data rather than as a broken environment*. Three distinct causes bit us: a build command that silently compiles only the root module and its imports, leaving sibling modules without object files; a C FFI target that fails to compile without a specific preprocessor definition, taking everything downstream with it; and a vendored dependency that goes missing or half-cloned. Together these cost roughly 3,700 valid challenges, 2,881 of them in a single repository. We report this because it is a reproducibility hazard for anyone building a compile-validated corpus, and because it means a low validation yield should be treated as a symptom to investigate rather than a property of the data.

167 3.6 Scoring and the outcome taxonomy

168 Solvers are scored by real compilation, and the outcome taxonomy distinguishes failures of the solver from failures of the harness or of the problem–model pairing (Table 1). Three outcome classes are excluded from the PASS denominator: malformed and trivial are properties of the problem, and context_exceeded is a property of the (problem, model) pair — the provider rejects the prompt, so the model never sees the problem, and scoring it zero would blame the solver for a challenge it was never shown. In our first baseline this affected 9% of the sample and, before it was flagged, silently depressed the reported rate.

175 harness_err — an attempt that ended in a transport-layer or provider failure rather than in an answer — is *kept* in the denominator and counted as a non-pass. This is the conservative choice, and it is the one the aggregator makes, so every rate we report is a lower bound in the presence of infrastructure noise. It is not free: one run in our grid hit a provider transport fault on a majority of its attempts, and §6 (L7) reports both the as-scored and the error-excluded figures for it rather than silently picking whichever is more flattering.

Table 1: Outcome taxonomy. The exclusions in the lower block are properties of the problem or of the (problem, model) pair, not of the solver. `tampered` is counted as a scorable non-pass, not an exclusion — it is a solver behaviour and §5.3 measures it directly. `harness_err` is also kept, which makes every reported rate a lower bound under infrastructure noise (§6, L7).

Outcome	Meaning	In denominator
<code>PASS</code>	compiles, hole-free, statements preserved	yes
<code>fail</code>	submitted a final answer that does not compile	yes
<code>turn_limit</code>	request budget exhausted while still working	yes
<code>gave_up</code>	solver invoked the give-up tool	yes
<code>tampered</code>	compiled by deleting or weakening a holed theorem	yes
<code>harness_err</code>	transport or provider failure; no answer delivered	yes
<code>malformed</code>	challenge does not compile (should not ship)	no
<code>trivial</code>	challenge and solution are identical	no
<code>context_exceeded</code>	prompt rejected; the model never saw the problem	no

Table 2: Corpus composition. Both holing strategies run over the same 26,157 candidate selections across 57 repositories; the released corpus is what survives two-sided compile validation. Concentration is high: a single repository supplies a third of the rows, which is why every headline number in this paper is macro-averaged per repository.

	Leaf holing	Whole-body holing	Total
Repositories	57	57	57
Mined candidates	26,157	26,157	—
Validated (released)	21,692	21,718	43,410
Validation yield	82.9%	83.0%	—
Largest repository share	33.8%	33.7%	—
Top-3 repository share	53.4%	53.3%	—

181 3.7 The tamper check

182 A compile-and-no-holes oracle has one exploit: make the file compile by removing or weakening
 183 the theorem you were asked to re-prove. We guard against this with a statement-preservation check.
 184 For Lean, the guard compares the exact statement text of each holed theorem up to the `:=` proof
 185 delimiter, so a weakened hypothesis or a deleted declaration is caught. For Rocq and Isabelle the
 186 proof delimiter is less uniform and the guard degrades to name-presence, which catches deletion but
 187 not weakening. Our corpus is Lean-only, so the strong version of the guard applies throughout; the
 188 weaker guarantee is flagged here for cross-prover extensions.

189 4 Dataset

190 4.1 Composition

191 The corpus is mined from 57 public Lean 4 repositories, each pinned to a specific revision and
 192 toolchain. Table 2 gives the totals. Of 26,157 mined candidate ablations per holing strategy, 21,692
 193 survive two-sided validation under leaf holing and 21,718 under whole-body holing, for a total of
 194 43,410 released challenges. The two strategies are applied to the same set of candidate selections, so
 195 the released splits are a matched pair in the sense of §3.4; the small difference in count arises because
 196 a deletion can validate under one holing strategy and not the other.

197 4.2 Provenance and pinning

198 A single tab-separated manifest is the source of truth for the corpus: repository name, language, URL,
 199 revision, checkout path, and toolchain. Source repositories are materialised from that manifest rather
 200 than vendored — they are multi-gigabyte once built — and every dataset manifest records the same

(repository, revision) pair that was ablated, so any released row traces back to an exact tree. The ablator additionally stamps the git remote and HEAD SHA onto each record.

4.3 Licensing

Licenses were surveyed at each repository’s *pinned revision* rather than at default-branch HEAD. This distinction is not pedantic: the platform API’s license-detection metadata only runs against the default branch, and silently degrades to a no-assertion value when queried at a specific commit, so trusting that field would have mis-surveyed every repository whose license file has ever changed. Our survey classifies the pinned-revision license text directly.

The corpus is genuinely mixed — Apache-2.0, MIT, BSD-3-Clause, LGPL-2.1, LGPL-3.0, Zlib, one non-standard non-commercial-only license, and four repositories with no license file at the pinned revision — so no single SPDX identifier is accurate in aggregate. The release carries other at the dataset level with a mandatory per-repository table (Appendix B). Five of the 57 repositories (8.8%) are flagged pending an explicit keep-or-drop decision: one under a non-commercial-only grant, and four with no license file at all, where absent an explicit grant default copyright applies and redistribution is not authorized. **[TODO: state the final keep/drop decision and, if any repos are dropped, the revised row counts in Table 2]**

4.4 Skew

The corpus is heavily concentrated. The largest repository contributes 33.8% of leaf rows and the three largest contribute 53.4%. Micro-averaged statistics over this corpus are therefore substantially statements about one repository. **All headline numbers in this paper are macro-averaged per repository**, and any per-repository comparison — especially the temporal analysis in §5.5, where repository age and repository size are correlated — is reported macro-averaged with the micro-average shown alongside for transparency.

4.5 Distribution and use

The corpus is released as two splits corresponding to the two holing strategies, keyed so that a consumer can join them on `challenge_id` to recover matched pairs. Each row carries its challenge text, ground-truth solution, deleted lemma and holed theorem names, provenance, and the proof-complexity features described in §5.6. Browsing the data requires nothing; *scoring* a prediction requires a built checkout of the source repository at its pinned revision, which is a real adoption barrier (§6, L3). Artifacts: [anonymized dataset].

5 Experiments

5.1 Setup

The solver is a ReAct-style agent with five tools: list the proof files in the work copy, read a file range, search the repository, submit a candidate solution, and give up. A submission is compiled immediately and the compiler’s output is returned to the agent as the tool result, so the loop is grounded in real feedback rather than self-assessment; every tool result also reports the remaining request budget. Runs are parameterised by a turn (request) budget, which §6 argues is a first-class experimental variable rather than an implementation detail.

The grid. We evaluate three models: `gpt-5.6-sol` (OpenAI), `claude-sonnet-5` (Anthropic), and `leanstral-1-5` (Mistral). The first two are general-purpose frontier models; the third is a Lean-specialised model, included so that capability and domain specialisation are not confounded into a single axis. The problem set is a matched sample: for each of the 57 repositories we draw up to two `challenge_ids` present in *both* splits under a fixed seed, giving 113 pairs (56 repositories contribute two, one contributes one). Every model is run over both holing strategies on this same identifier set at a 50-turn budget — six runs, 678 solve attempts in total — so each model sees each deletion twice, once with the surrounding proof skeleton surviving and once without.

Ten of the 113 pairs are malformed on both sides and are excluded from both, leaving 103 scorable pairs; these are repository-environment gaps of exactly the kind §3.5 describes, not ablator defects,

and are reclaimable with targeted builds. Four repositories lose all of their sampled problems this way, so 53 repositories enter every macro-average. Rates are macro-averaged per repository (§4.4) with the micro-average reported alongside, and confidence intervals are bootstrap intervals over the resampled unit (repositories for macro, problems for micro).

Cost. The six runs consumed 441.2M input and 10.4M output tokens and 95.3 hours of aggregate solver time (the sum of per-problem wall-clock; the runs themselves were parallelised). Per-model turn and token costs are in Table 5. All compilation ran on a single workstation; no accelerators were used.

Reference run. An earlier baseline motivated this design and is reported here for that reason only, not as a headline: a small Lean-tuned model on a 100-problem leaf-holing sample across 49 repositories at a 50-turn budget scored 38/91 scorable (42%), with `turn_limit` occurring 44 times — more often than PASS — and nine problems (9%) exceeding the model’s 262,144-token context. The grid below shows that the modal outcome is strongly model-dependent, which that single run could not have revealed.

5.2 Main results

Table 3 gives the pass rates and Table 4 the full outcome composition underneath them; Figure 1 shows the same composition as shares of the 113 attempts per (model, strategy). Two things are visible, and the second is the more important one.

The benchmark discriminates — between tiers, not between every pair. `gpt-5.6-sol` separates cleanly from the other two models and no model is near ceiling or floor: 49.1% / 29.2% / 30.8% macro PASS under leaf holing, and 47.2% / 28.3% / 28.3% under whole-body. `claude-sonnet-5` and `leanstrat-1-5` are statistically indistinguishable at this budget — a finding in itself, since one is a frontier general-purpose model and the other a far smaller Lean-specialised one. (An earlier draft of this grid reported `leanstrat` at 17.9%/24.0%; that arm was degraded by provider transport faults, and the clean re-run reported here erases the gap — §6, L7.) The corpus is usable as an eval today rather than only as a training pool.

The strongest model never fails honestly. Of `gpt-5.6-sol`’s 103 scorable leaf problems, 51 are genuine passes and 50 are caught tamperers — the file compiles, has no holes, and no longer contains the theorem it was asked to re-prove. The remaining two attempts are provider connection errors. There are *zero* turn-limit outcomes, *zero* give-ups and *zero* submitted-but-non-compiling answers, under either strategy. Every attempt this model made either re-derived the lemma or attacked the statement.

The consequence for benchmark design is direct. A scorer that checks only “compiles and contains no holes” — which is what a compile-based oracle degenerates to without a statement-preservation check — would have reported 101/103 (98.1%) on leaf holing and 102/103 (99.0%) on whole-body holing, against true rates of 49.5% and 47.6%. The guard of §3.7 is not hygiene; on this model it is the difference between a 98% result and a 50% one.

5.3 Reward hacking under a verified reward (#136)

A verified reward has no false positives on the *proof*: a memorized proof that compiles is a correct proof. Its attack surface is the *statement*, and on this grid that surface is attacked constantly.

Tamper rate ranks with capability. Measured as a share of scorable attempts (Figure 2), the guard fires on 48.5% / 51.5% of `gpt-5.6-sol`’s leaf / whole-body attempts, 26.2% / 25.2% of `claude-sonnet-5`’s, and 26.8% / 22.0% of `leanstrat-1-5`’s. The model that passes most also tampers most, by roughly a factor of two over the other two — which tamper at statistically indistinguishable rates, mirroring their indistinguishable pass rates. We report this as an observation on three models at one budget, not as a law; what it does establish is that the behaviour is not a quirk of one weak model, and that a solver’s tamper rate tracks its capability tier rather than its architecture or specialisation.

Table 3: Main grid. 113 matched pairs over 57 repositories, 103 scorable after excluding the 10 pairs that are malformed on both sides (leanstral additionally loses 6 leaf and 3 whole-body attempts to context_exceeded). Macro is the mean of per-repository rates over the 53 repositories with at least one scorable problem; intervals are bootstrap 95% intervals. “Tamper” is the share of scorable attempts caught by the statement-preservation guard, and “compile-only” is the rate a scorer without that guard would have reported.

Model	Strategy	Macro PASS		Micro PASS		Tamper	Compile-only
gpt-5.6-sol	leaf	49.1%	[43.4, 54.7]	49.5% (51/103)	[40.8, 59.2]	48.5%	98.1%
gpt-5.6-sol	whole	47.2%	[42.5, 51.9]	47.6% (49/103)	[38.8, 57.3]	51.5%	99.0%
claude-sonnet-5	leaf	29.2%	[23.6, 34.9]	29.1% (30/103)	[20.4, 38.8]	26.2%	55.3%
claude-sonnet-5	whole	28.3%	[22.6, 34.0]	29.1% (30/103)	[20.4, 37.9]	25.2%	54.4%
leanstral-1-5	leaf	30.8%	[26.0, 36.5]	30.9% (30/97)	[21.6, 41.2]	26.8%	57.7%
leanstral-1-5	whole	28.3%	[22.6, 34.0]	29.0% (29/100)	[20.0, 39.0]	22.0%	51.0%

Table 4: Outcome composition, counts out of the 113 attempts per (model, strategy). The rightmost two columns are excluded from the PASS denominator; the rest — including harness transport faults, which stay in as non-passes (§6, L7) — are not. The mix differs far more across models than the pass rate does: gpt-5.6-sol’s entire non-pass mass is tampered, while leanstral-1-5 carries the grid’s only residual provider transport faults (§6, L7).

Model	Strategy	PASS	tamp.	fail	turn-lim.	gave up	harness	ctx.	malf.
gpt-5.6-sol	leaf	51	50	0	0	0	2	0	10
gpt-5.6-sol	whole	49	53	0	0	0	1	0	10
claude-sonnet-5	leaf	30	27	19	18	2	7	0	10
claude-sonnet-5	whole	30	26	17	25	2	3	0	10
leanstral-1-5	leaf	30	26	1	16	1	23	6	10
leanstral-1-5	whole	29	22	0	18	2	29	3	10

297 **How the statement is attacked.** The guard distinguishes two cases. Removing the holed theorem’s
 298 declaration outright accounts for 83 of gpt-5.6-sol’s 103 tampers across both strategies, 49 of 53
 299 for claude-sonnet-5, and 47 of 48 for leanstral-1-5; keeping a declaration whose statement
 300 no longer matches the original accounts for the remaining 20, 4 and 1. Deletion is thus the dominant
 301 attack, which matters for cross-prover extensions: the weaker name-presence guard available for
 302 Rocq and Isabelle (§3.7) would still catch the large majority of these, though not all.

303 **Budget dependence: the hypothesis is refuted, in both directions.** We predicted, from the
 304 reference run (2 tampers at 30 turns rising to 4 at 50), that tamper rate would rise with turn budget —
 305 more budget, more opportunity to find the exploit. The full curve — genuine re-runs of the matched
 306 sample at announced budgets of 15, 30 and 100 turns alongside the 50-turn grid (Figure 3) — does
 307 not show that. Summed over both strategies, claude-sonnet-5’s tamper count rises only mildly
 308 with budget (42 / 50 / 53 / 56 at 15/30/50/100 turns), alongside a rising pass count; gpt-5.6-sol’s
 309 falls mildly (107 / 103 / 103 / 99), and peaks at the lowest budget. The strongest model tampers most
 310 when it can least afford a real derivation: budget *starvation*, not budget surplus, is what pushes it
 311 toward the exploit, and extra budget converts would-be tampers into genuine solves. The reason split
 312 is stable across every point — outright statement deletion accounts for at least 94% of tampers at
 313 every budget, for both models — so the cross-prover implication of §3.7 is unchanged along the
 314 whole curve.

315 **Why this belongs in a paper about verified code generation.** The measurement is possible only
 316 because the reward is a kernel and the guard is separable from it. It is also an argument for the
 317 corpus as a reinforcement-learning environment: it demonstrates concretely which guards such an
 318 environment needs, and that omitting them yields a reward signal that a capable policy saturates for
 319 the wrong reason.

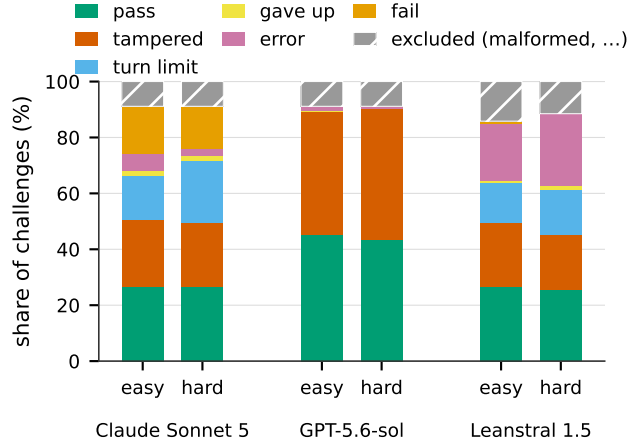


Figure 1: Outcome composition per (model, strategy), as shares of the 113 attempts (counts in Table 4). malformed is hatched: it is excluded from the PASS denominator, unlike tampered, which is a solver behaviour and stays in. The mix — not the pass rate — is where the models differ most: gpt-5.6-sol’s non-pass mass is entirely tampered (§5.3).

Table 5: Cost of the grid, both strategies combined. Average turns is taken over attempts that reached a solver outcome, so malformed and transport-fault rows — which never consumed budget — are excluded from that column only.

Model	Avg. turns	Input (Mtok)	Output (Mtok)	Solver time (h)
gpt-5.6-sol	8.0	38.4	1.64	9.2
claude-sonnet-5	38.8	281.4	4.45	21.0
leanstral-1-5	28.6	121.3	4.35	65.1
Total	—	441.2	10.44	95.3

5.4 Do the two holing strategies differ? (#129)

We publish leaf and whole-body holing as separate easy and hard splits on the premise that the second is strictly harder: with no surviving proof skeleton, the solver must re-derive from the statement alone. This section tests that premise on the matched sample, using exact McNemar tests on the discordant pairs — the correct test for a paired design.

The pass-rate gap is null for both general-purpose models. For gpt-5.6-sol, 6 problems pass under leaf holing only and 4 under whole-body only ($p = 0.754$); for claude-sonnet-5, 6 and 6 ($p = 1.000$). The macro rates differ by 1.9 and 0.9 percentage points, well inside the intervals in Table 3. We state this plainly rather than hedging it: at a 50-turn budget these two splits are not ordered by difficulty for any model in the grid (the specialist’s apparent reversal is dealt with below), and the easy/hard naming is not supported by the pass rates. A problem a model cannot solve without the skeleton it usually cannot solve with it either.

What the strategies do change is the failure composition and the turn cost. For claude-sonnet-5 the PASS count is identical (30 and 30) while turn_limit rises from 18 to 25 and fail drops from 19 to 17: the same problems are solved, but the model spends longer and more often runs out of budget still working rather than delivering a wrong answer. Average turns rise under whole-body holing for all three models (7.7 to 8.3, 37.1 to 40.5, and 27.0 to 30.4), which is the one effect of the strategy that is consistent across the grid. For gpt-5.6-sol the composition shift shows up as tampering instead, 50 to 53. The hypothesis that whole-body holing moves probability mass toward turn_limit is therefore supported; the hypothesis that it lowers PASS is not.

The specialist’s apparent reversal was an artifact, and the clean re-run erased it. An earlier draft of this grid reported leanstral-1-5 going the other way — 17.9% on leaf holing against

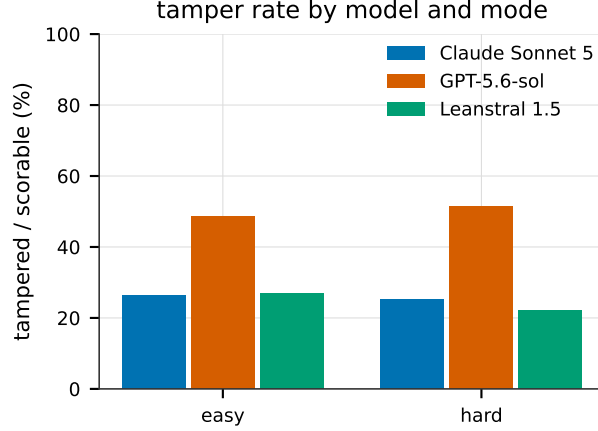


Figure 2: Share of scorable attempts caught by the statement-preservation guard, per model and holing strategy, at the 50-turn budget. The ordering matches the pass-rate ordering in Table 3: the more capable the solver, the larger the fraction of its attempts that are caught games rather than honest failures.

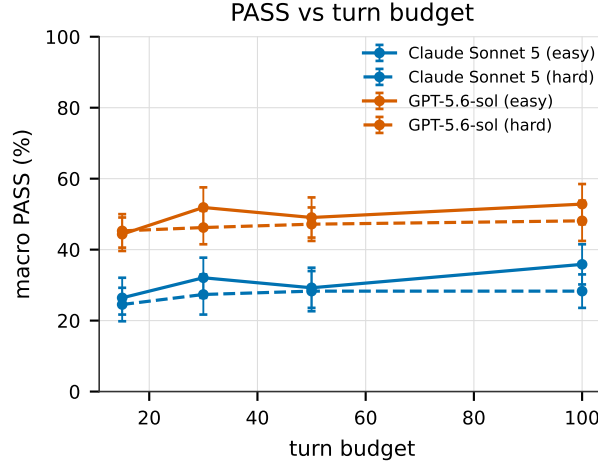


Figure 3: Macro PASS against the announced turn budget (15/30/50/100), per model and holing strategy, bootstrap 95% intervals. `gpt-5.6-sol` saturates by 30 turns; `claude-sonnet-5` does not saturate on leaf holing by 100 (35.8% at 100 against 29.2% at 50, with its turn-limit share falling from 37% to 8% across the four points). Each point is a genuine re-run at that announced budget — post-hoc thresholding of the 100-turn arm on `turns_used` is *not* equivalent (§6, L2).

342 24.0% on whole-body ($p = 0.118$) — and flagged the result as suggestive rather than established,
 343 because that arm had lost 60 of 103 leaf attempts against 44 of 100 whole-body to provider transport
 344 faults: an asymmetric confound, and a clean re-run was named the cheapest outstanding item in
 345 the grid. We ran it (re-solving exactly the faulted attempts at low request concurrency, splicing by
 346 challenge id). The reversal does not survive: 30.8% leaf against 28.3% whole-body, 5 discordant pairs
 347 each way, $p = 1.000$ — the same null as both general-purpose models. The methodological point
 348 stands with the sign flipped: an infrastructure asymmetry of that size can *manufacture* an apparent
 349 model-specific effect, which is why the outcome taxonomy separates transport faults from solver
 350 behaviour (Table 1) instead of folding them into failures.

351 **Consequence for the release.** We continue to publish both splits, since they are genuinely different
 352 tasks with different failure compositions and turn costs, but the *easy/hard* labels overstate what we
 353 have shown — no model orders them at this budget. The honest description is “leaf-step holing” and
 354 “whole-body holing”, which is the terminology used throughout this paper.

355 5.5 Contamination, measured

356 Every source repository is public, so contamination is the objection this dataset must answer directly.
357 We structure the answer around a distinction the literature tends to merge.

358 **Instance-level memorization** — the model saw *this exact problem* — is defeated by construction.
359 The sliced file, with these specific lemmas deleted and these specific downstream uses holed, never
360 existed as a document. This is a stronger claim than a git-history-mining approach can make, where
361 the challenge is a real historical commit and the solution is the literal next commit, both present in
362 training data *as a pair*.

363 **Knowledge-level memorization** — the model knows *this lemma* — is not defeated, and we do not
364 claim otherwise. What we can do is measure where the first stops mattering and the second takes
365 over, with four instruments, two of which have landed.

366 5.5.1 Corpus membership against a dated snapshot (#137)

367 The Stack v2 [10] is built by traversing a Software Heritage [3] graph snapshot with a documented
368 date (2023-09-06); The Stack v1 [8] was crawled between November 2021 and June 2022. Software
369 Heritage publishes, per origin, the dated history of every crawl it has performed. This turns “is this
370 repository in the pretraining corpus?” from a guess into a query: we resolve each of the 57 source
371 repositories against the Software Heritage API and compare its *earliest* recorded visit against those
372 two cutoffs.

373 All 57 resolved. Forty-six have no recorded Software Heritage visit at all; nine were first visited only
374 after the Stack v2 snapshot; **two** — `starkware-formal-proofs` and `symcrust` — were visited
375 before it, and only `symcrust` (a vendored copy of a Microsoft cryptographic library, earliest visit
376 2022-03) also clears the Stack v1 cutoff. An early visit is necessary but not sufficient for inclusion,
377 since the Stack’s own license and deduplication filters run afterwards, so we report `likely_in` rather
378 than a hard yes. The conclusion is nonetheless strong in the direction that matters: 55 of 57 source
379 repositories were not crawled before either snapshot, so Stack-derived pretraining cannot be the route
380 by which a model knows these specific developments. A secondary cross-check against infini-gram
381 slug counts in Dolma v1.7 returned zero for every repository except `lean-mlir` (2) and `symcrust`
382 (3), which is consistent but too sparse to carry weight on its own.

383 This instrument replaces repository popularity, which we had used first and now report only as a
384 robustness footnote for the reasons in §5.5.5.

385 5.5.2 Temporal holdout by lemma introduction date (#132)

386 Repository age is the wrong unit: a repository created in 2021 can contain a lemma written in 2026.
387 We therefore date the *deleted lemma* in every challenge, by walking the pinned checkout’s history for
388 the commit that first introduced an actual declaration of that name (not merely a mention of it). This
389 succeeds for **21,692 of 21,692** deleted lemmas in the leaf split, across all 57 repositories — complete
390 coverage, which matters because a dating procedure that silently fails on the oldest lemmas would
391 manufacture exactly the null we are about to report. The corpus is sharply recent by this measure:
392 only 430 lemmas (2.0%) predate 2024-06 and only 1,164 (5.4%) predate 2025-01.

393 Partitioning the grid’s problems at those two dates gives Table 6, and the pattern splits cleanly by
394 model class. For the two general-purpose models there is no systematic advantage on the lemmas
395 they could have seen: `claude-sonnet-5` is directionally *worse* on the older lemmas in all four of
396 its cells, and `gpt-5.6-sol` is mixed (higher in three cells, lower in one). For the Lean-specialised
397 `leanstral-1-5` the direction is consistent: the pre-cutoff side is higher in all four of its cells (41.7%
398 against 27.6% at the 2024-06 cutoff on leaf holing; 50.0% against 25.5% on whole-body), exactly the
399 signature knowledge-level memorization predicts, in exactly the model whose training corpus is most
400 concentrated on Lean.

401 The caveat is severe and we state it rather than burying it: the pre-cutoff side contains 10 problems
402 at the 2024-06 cutoff and 19 at 2025-01, out of 103, so no single cell is individually significant —
403 what is suggestive is the consistency of the direction across all four specialist cells against its absence
404 in the eight generalist ones. The test is underpowered *because* the corpus is recent — the same

Table 6: Temporal holdout. Macro-averaged PASS on problems whose deleted lemma was introduced before versus after each cutoff. n is the number of problems on each side, out of 103 scorable. The pre-cutoff side is small by construction: only 2.0% of the corpus’s deleted lemmas predate 2024-06. Neither general-purpose model is consistently better on the older lemmas; the Lean-specialised model is better on them in all four of its cells.

Model	Strategy	Cutoff 2024-06 ($n = 10/93$)		Cutoff 2025-01 ($n = 19/84$)	
		pre	post	pre	post
gpt-5.6-sol	leaf	50.0%	49.0%	45.5%	50.0%
gpt-5.6-sol	whole	58.3%	45.9%	50.0%	46.6%
claude-sonnet-5	leaf	16.7%	31.6%	22.7%	31.8%
claude-sonnet-5	whole	25.0%	29.6%	18.2%	31.8%
leanstral-1-5	leaf	41.7%	27.6%	31.8%	28.4%
leanstral-1-5	whole	50.0%	25.5%	36.4%	26.1%

property that makes the membership result above come out the way it does. The honest summary: no contamination signal for the general-purpose models with either instrument, and a directionally consistent, underpowered retrieval signal for the domain specialist — which the verbatim recall probe (§5.5.4) is the instrument built to adjudicate, since it measures memorization directly rather than through pass rates.

5.5.3 Deletion-count sweep (#133)

Deletion count is the dial that separates the two kinds of memorization. Reconstructing one lemma is plausibly retrieval; reconstructing five interlocking lemmas that were *never simultaneously absent* from any document is not. We sweep the number of deleted lemmas over $\{1, 2, 3, 5\}$ on a fixed problem set, holding the corollary and repository distribution constant across depths so that the curve is about depth rather than about which problems were drawn, and report the pass-rate decay with confidence intervals. The shape is the finding: a sharp cliff between one and two deletions would indicate that retrieval was carrying the single-deletion case. This is a perturbation-robustness measurement in the GSM-Symbolic tradition, and a methodological contribution rather than merely a defense.

The lemma dating above makes this the sharper of the two remaining instruments: with the temporal axis this compressed, depth is the dimension along which instance-level and knowledge-level memorization actually come apart in this corpus.

Result. We ran the sweep on claude-sonnet-5 at the 50-turn budget: 100 problems per depth over 30 repositories, drawn from the paired sample’s files with the corollary pinned by the mining seed, so every depth ablates *the same 100 (file, corollary) pairs* — the distribution is constant across depths by construction, and verified: a tuple enters the set only if it yields exactly N deletions *and* compile-validates on both sides at every depth. Figure 4 shows the decay: 27.1% macro PASS at one deletion, 15.6% at two, 15.1% at three, 11.5% at five (micro: 24% / 8% / 6% / 5%).

The shape carries the interpretation, and it is not the retrieval story. Against an independence reference — treating a depth- N problem as N independent single-deletion problems, p_1^N with $p_1 = 0.24$ — the one-to-two drop is roughly what independence predicts (8% observed, 5.8% predicted), while depths three and five sit far *above* it (5% observed against 0.08% predicted at depth five). Deleting five interlocking lemmas from one closure is much easier than five independent problems: the lemmas share context, statements, and often a proof idiom, and re-deriving one supplies material for the next. For this model the sweep therefore shows a conjunction-difficulty curve, not a memorization cliff — consistent with the temporal holdout, which finds no pre-cutoff advantage for this model either (Table 6); the two instruments agree. The model whose temporal cells *do* all point toward retrieval (leanstral-1-5, above) is the natural target for a follow-up sweep. Tampering, incidentally, falls with depth (35 / 27 / 20 / 20 of 100): deleting the statement of five theorems at once is apparently a less tempting exploit than deleting one.

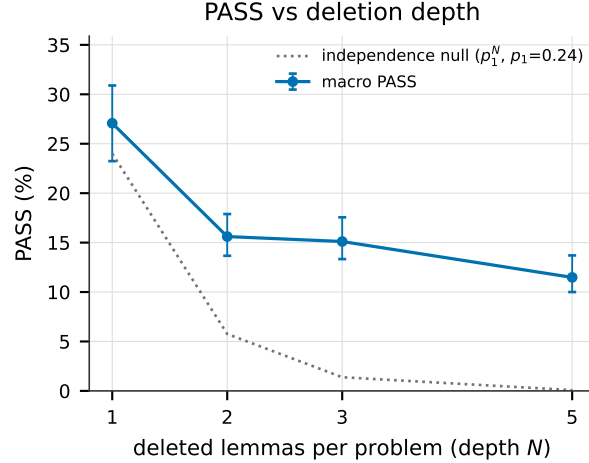


Figure 4: Macro PASS against deletion depth $N \in \{1, 2, 3, 5\}$ on a fixed 100-problem set (same files, same seed-pinned corollaries at every depth; each problem compile-validated at every depth), `claude-sonnet-5`, 50-turn budget, bootstrap 95% intervals. The dotted reference is the independence null p_1^N : the one-to-two drop roughly matches it; the tail sits far above it. The curve is a conjunction-difficulty measurement, not a memorization cliff.

Table 7: Verbatim recall probe (#134). 113 deleted lemmas per model; “nt” is the non-trivial subset (proof bodies over one line, $n = 96$). Bands: high ≥ 0.9 normalised Levenshtein similarity.

Model	exact	mean lev	mean tokF1	high band	exact (nt)	high band (nt)
<code>claude-sonnet-5</code>	0/113	0.286	0.327	0.9%	0/96	0.0%
<code>gpt-5.6-sol</code>	2/113	0.269	0.330	1.8%	0/96	0.0%
<code>leanstrat-1-5</code>	0/113	0.189	0.218	0.0%	0/96	0.0%

5.5.4 Verbatim recall probe (#134)

The correlational tests above cannot separate *memorization* from *ability*: a model that solves more might simply be better. A recall probe can. For each deleted lemma in the paired sample we prompt the model with the lemma name and surrounding file context, body removed, and ask it to state the lemma; we score by exact match after whitespace normalisation, normalised Levenshtein similarity, and token-level F1 (exact match alone will be near-zero and misleading; similarity bands are high ≥ 0.9 , mid ≥ 0.6 , low otherwise). The two splits delete the *same* lemma (verified byte-identical across all 113 pairs), so each lemma is probed once per model and the score joined to both splits’ outcomes; 339 probes total, all completed without error. One stratification is load-bearing: a one-line `simp [foo]` is reproduced by any model that knows Lean with zero memorization, so every rate is also reported over the *non-trivial* subset (proof bodies of more than one line, 96 of 113).

Verbatim recall is at the floor (Table 7). No model reproduced a single multi-line proof body: 0/96 exact on the non-trivial subset for all three. The only two exact matches in 339 probes — both by `gpt-5.6-sol` — are one-line `simp` calls whose argument is the definition named in the statement, which is competent Lean, not recall; the single high-band non-exact hit is likewise a one-liner. Mean similarity of 0.19–0.29 is what surface overlap of any two proofs of the same statement looks like. This is the result the membership check of §5.5.1 predicts: with 55 of 57 repositories never crawled before either Stack snapshot, there is little for the models to have memorized. Two independent instruments, one direct on the corpus and one direct on the models, agree.

The payoff is the correlation between recall score and solve outcome on the *same* problems (Table 8): point-biserial r between similarity and PASS ($n = 103$ scorable per cell; `tampered` counted as fail), with a 5,000-sample bootstrap CI, a 10,000-draw permutation p , and a partial correlation controlling for log proof length, since short proofs are both easier to guess and easier to solve. For `gpt-5.6-sol` and `leanstrat-1-5`, recall and solve success are uncorrelated ($|r| \leq 0.03$, $p \geq 0.73$): their pass rates are not retrieval. For `claude-sonnet-5` there is a small positive association, significant on

Table 8: Recall–solve correlation, per model per split. Point-biserial r between normalised Levenshtein recall similarity and PASS on the same challenge; bootstrap 95% CI; permutation p ; partial r controls for log proof length.

Model	Split	$r(\text{lev, PASS})$	95% CI	perm. p	partial r
claude-sonnet-5	easy	+0.184	$[-0.009, +0.375]$	0.061	+0.171
claude-sonnet-5	hard	+0.257	$[+0.056, +0.451]$	0.007	+0.235
gpt-5.6-sol	easy	+0.033	$[-0.156, +0.241]$	0.738	+0.097
gpt-5.6-sol	hard	+0.015	$[-0.172, +0.219]$	0.880	+0.111
leanstrat-1-5	easy	+0.034	$[-0.220, +0.290]$	0.728	+0.041
leanstrat-1-5	hard	+0.020	$[-0.221, +0.247]$	0.840	+0.028

hard ($r = +0.257$, $p = 0.007$; +0.235 size-controlled) and marginal on easy ($r = +0.184$, $p = 0.061$); with six cells tested the surviving cell is $p \approx 0.04$ Bonferroni-corrected — real but weak, about 7% of variance. The direction is the one the open question above predicts: the whole-body split, where no proof skeleton constrains the reconstruction, is where what the model knows about the lemma has more work to do. Crucially, $r = +0.26$ against a floor-level recall distribution is not a contamination finding: nothing was recalled verbatim, so the association is between *slightly less bad* reconstructions and solving — what one expects if both are downstream of the model having a good idea how the proof goes, i.e. ability, not memorization. A contamination reading would require recall mass at the top of the range, and there is none. Two caveats bound the claim: absence of verbatim recall does not rule out knowledge-level memorization (a model can know a lemma’s content without its surface form), and the cells are powered to detect $r \approx 0.28$ at 80%, so a genuinely small effect would be missed.

5.5.5 What we do not use

We deliberately do not use repository popularity as a contamination proxy. Popularity proxies contamination only insofar as it proxies duplication count in the training corpus, which is the variable memorization actually scales with [2], and in this corpus that chain is broken: 41% of the 57 repositories were created after 2025-06 and nine of the nineteen with at least 100 stars postdate mid-2024, so “popular” and “plausibly in the training set” are substantially decoupled here. Popularity is also not a standard instrument in the contamination literature. It was the first thing we tried, and its qualitative direction agreed with the membership check above; having a direct instrument, we drop it rather than report a weak proxy alongside a strong one.

5.6 A calibrated difficulty model (#135)

Benchmark papers rarely ship a predictor of their own difficulty. We do, and we report it honestly, which means reporting calibration and not only discrimination.

Features are computed *inside* the ablator, so they are visible in the released rows rather than being a post-hoc artifact of our analysis code: size and shape (line count, tactic count, cyclomatic complexity, fan-in of the deleted lemma), what the proof *does* (counts of automation, rewriting and structural steps, an automation-only flag, maximum nesting depth), and what the corollary rests on (direct and transitive dependency counts). Size alone cannot distinguish a one-line simp call from a forty-line induction with the same step count, and that distinction is most of what decides whether a lemma can be re-derived.

The model is trained on the grid outcomes of §5.2 — 678 labelled attempts, of which 615 are scorable — then evaluated on a **disjoint** second sample that is scored *before* its outcomes exist, a discipline the pipeline enforces rather than merely recommends. We report held-out ROC-AUC for discrimination and the Brier score with the Murphy decomposition (reliability, resolution, uncertainty) plus a decile reliability diagram for calibration, and the per-feature coefficients, since “fan-in and tactic count predict difficulty, file size does not” is a more useful statement to this audience than an AUC.

Three caveats the numbers carry. First, the labels are budget-dependent: this is difficulty *at budget B*. Second, the labels are *model*-dependent in a way the grid makes concrete — §5.4 found that the harder of the two holing strategies is harder for two models and easier for a third, so a single

506 difficulty score per problem is already an approximation. Third, the scorer counts tampered and
507 `turn_limit` as failures while dropping malformed, trivial and `context_exceeded`, matching
508 Table 1; since roughly half of `gpt-5.6-sol`’s attempts are tampers, the model is in part predicting
509 where a solver will choose to cheat.

510 **[TODO: #135: held-out AUC, Brier with Murphy decomposition, reliability diagram (Fig. 7),**
511 **feature coefficients; cross-mode transfer (train on leaf, score whole-body) attempted or explicitly**
512 **deferred with a reason]**

513 6 Limitations

514 **L1: Lean only.** Every released challenge is Lean 4. The ablator exists for Rocq and Isabelle and
515 the record schema is shared, but the Rocq corpus is unmined and the Isabelle corpus is only partially
516 mined: of 19,018 challenges mined from the seL4 verification development, 14,594 sit under the
517 refinement proof, whose Isabelle sessions require a design specification that is *generated* from seL4’s
518 Haskell model and is absent from a plain checkout. Until that generation step runs in our pipeline,
519 only sessions below the design spec can be validated, and we ship none of them here. Nothing in this
520 paper should be read as a claim about cross-prover generality.

521 **L2: Pass rates are budget-dependent, and how they depend on budget is model-dependent.**
522 `turn_limit` was the modal outcome in our first baseline, but the grid shows it is not a property of
523 the task: summed over both strategies at 50 turns it accounts for 0 of `gpt-5.6-sol`’s 206 scorable
524 attempts, 43 of `claude-sonnet-5`’s 206, and 20 of `leanstrat-1-5`’s 203. Average turns also rise
525 under whole-body holing for every model (§5.4), so the budget interacts with the strategy. Every
526 pass rate we report is a pass rate at a stated budget and should be read as a lower bound at that
527 budget rather than as a property of the model. This propagates: the difficulty model of §5.6 predicts
528 difficulty-at-budget, and any contamination gap measured on top of a budget-limited baseline inherits
529 the same confound. The curve of Figure 3 quantifies this: `claude-sonnet-5`’s leaf-holing rate is
530 still rising at 100 turns (26.4% / 32.1% / 29.2% / 35.8% at 15/30/50/100) while its whole-body rate is
531 flat from 30, and `gpt-5.6-sol` is saturated by 30 turns on both. The curve also cannot be obtained
532 cheaply: deriving it from the 100-turn arm by post-hoc thresholding on `turns_used` under-predicts
533 `claude-sonnet-5`’s genuine 15-turn passes by more than $3\times$ (8 predicted, 27 measured; exact
534 binomial $p < 0.001$ on the discordant pairs, in all four sonnet cells), because the agent is told its
535 budget and paces itself to the announcement — an 8-turn solver like `gpt-5.6-sol` shows no such
536 gap. Budget curves for models that pace themselves require genuine per-budget runs.

537 **L3: Scoring requires a built checkout.** The data can be browsed by anyone; scoring a prediction
538 requires the repository’s prebuilt `.olean` closure at the pinned revision, at roughly 5–7 GB per
539 Mathlib-dependent repository and on the order of a terabyte for all 57. We publish prebuilt closures
540 for a small demo subset sufficient for a quickstart, not for the full corpus: five repositories —
541 `verified-compiler`, `LNSym`, `capless-lean`, `FVIntmax`, and `ArkLib` — as content-addressed
542 `tar.zst` archives at the pinned revisions, about 5.9 GB compressed (17 GB unpacked) in total. This
543 is the single largest adoption barrier we know of, and it is a property of the release, not of the method.

544 **L4: Corpus skew.** One repository supplies 33.8% of leaf rows and the top three supply 53.4%. We
545 macro-average everywhere, which mitigates the problem but does not remove it: the tail repositories
546 contribute few problems each, so per-repository estimates in the tail are noisy, and any analysis
547 correlated with repository size inherits that noise structure.

548 **L5: Context overflow.** Roughly 10% of challenges exceeded a 262,144-token context window in
549 our first baseline, even after minimal slicing. We flag these `context_exceeded` and exclude them
550 from the denominator, since the provider rejects the prompt and the model never sees the problem.
551 Excluding them is the honest accounting, but it also means the largest problems — plausibly among
552 the hardest — are systematically unmeasured, and the exclusion set varies by model, so cross-model
553 comparisons are over slightly different problem sets. The grid’s macro-balanced sample is much less
554 affected (3 exclusions out of 678 attempts, all in one model’s whole-body run, Table 4), which is
555 itself a reminder that the exclusion rate is a property of the sampling scheme and not of the corpus.

556 **L6: Unresolved licensing on part of the corpus.** Five of 57 repositories are flagged in §4.3
557 pending an explicit keep-or-drop decision. Until that is settled, the exact released row count is
558 provisional.

559 **L7: One grid arm needed a transport-fault re-run, and some faults persist.** The original
560 `leanstrat-1-5` arm lost 60 of 103 leaf attempts and 44 of 100 whole-body attempts to TLS-layer
561 and read-timeout failures against the provider — concentrated enough, and asymmetric enough
562 across the two strategies, to manufacture the apparent easy/hard reversal discussed in §5.4. The
563 faulted attempts were re-solved at low request concurrency and spliced by challenge id; the numbers
564 throughout this paper are from the re-run. Even so, 23 leaf and 29 whole-body attempts still ended in
565 transport faults, and following Table 1 the aggregator keeps them in the denominator as non-passes,
566 so the `leanstrat` row of Table 3 remains a mild lower bound: dropping the residual faulted attempts
567 gives 39.4% and 40.4% over 47 repositories on each strategy. The two general-purpose arms are
568 essentially unaffected (7, 3, 2 and 1 faults out of 113).

569 **L8: One sample, one seed.** Every number in §5 comes from a single 113-pair draw at seed 42,
570 run once per (model, strategy, budget) — the main grid at 50 turns, the budget curve of Figure 3 at
571 15/30/100 on the same draw. There is no across-seed replication and no repeated sampling of the
572 same problem, so the intervals we report capture sampling variation over problems and repositories
573 but not the solver’s own run-to-run variance. Given that these solvers are stochastic and agentic, that
574 component is not negligible.

575 7 Conclusion

576 Proof repair inside a real development is a different task from proving a competition statement, and it
577 is closer to what proof engineering actually is. We show it can be posed at scale by syntactic ablation,
578 and that the resulting problems can be validated on both sides by the compiler — 43,410 of them,
579 from 57 pinned Lean 4 repositories. The validation discipline is the part we would most like to see
580 adopted elsewhere: it is cheap relative to the mining, it catches unwinnable problems that no amount
581 of downstream evaluation would surface, and it turns “is this benchmark well-formed?” from an
582 assertion into a build artifact.

583 The result we did not expect is the one we would most like the field to take seriously. On this task
584 the strongest model we evaluated does not fail; it cheats. Every scorable attempt it made either
585 re-derived the lemma or made the file compile by removing the theorem it was asked to prove, and
586 a scorer without a statement-preservation check would have called that 98% success. A verified
587 reward is sound about proofs and says nothing about statements, and at current capability levels
588 the gap between those two is not a corner case — it is half the results table. Benchmarks and RL
589 environments built on proof checkers need the second half of the oracle written down, published, and
590 measured, not assumed.

591 References

592 References

- 593 [1] Azerbayev, Z., Piotrowski, B., Schoelkopf, H., Ayers, E.W., Radev, D. & Avigad, J. (2023) ProofNet:
594 Autoformalizing and formally proving undergraduate-level mathematics. *arXiv:2302.12433*.
- 595 [2] Carlini, N., Ippolito, D., Jagielski, M., Lee, K., Tramer, F. & Zhang, C. (2023) Quantifying memorization
596 across neural language models. In *International Conference on Learning Representations (ICLR)*.
- 597 [3] Di Cosmo, R. & Zacchiroli, S. (2017) Software Heritage: Why and how to preserve software source code.
598 In *International Conference on Digital Preservation (iPRES)*.
- 599 [4] Golchin, S. & Surdeanu, M. (2024) Time travel in LLMs: Tracing data contamination in large language
600 models. In *International Conference on Learning Representations (ICLR)*.
- 601 [5] Han, J.M., Rute, J., Wu, Y., Ayers, E.W. & Polu, S. (2022) Proof artifact co-training for theorem proving
602 with language models. In *International Conference on Learning Representations (ICLR)*.

- [6] Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K. & Stoica, I. (2025) LiveCodeBench: Holistic and contamination-free evaluation of large language models for code. In *International Conference on Learning Representations (ICLR)*.
- [7] Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. & Narasimhan, K. (2024) SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*.
- [8] Kocetkov, D., Li, R., Ben Allal, L., Li, J., Mou, C., Muñoz Ferrandis, C., Jernite, Y., Mitchell, M., Hughes, S., Wolf, T., Bahdanau, D., von Werra, L. & de Vries, H. (2023) The Stack: 3 TB of permissively licensed source code. *Transactions on Machine Learning Research*.
- [9] Liu, C., Shen, J., Xin, H., Liu, Z., et al. (2023) FIMO: A challenge formal dataset for automated theorem proving. *arXiv:2309.04295*.
- [10] Lozhkov, A., Li, R., Ben Allal, L., et al. (2024) StarCoder 2 and The Stack v2: The next generation. *arXiv:2402.19173*.
- [11] Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S. & Farajtabar, M. (2025) GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations (ICLR)*.
- [12] Tsoukalas, G., Lee, J., Jennings, J., Xin, J., Ding, M., Jennings, M., Thakur, A. & Chaudhuri, S. (2024) PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- [13] Yang, K. & Deng, J. (2019) Learning to prove theorems via interacting with proof assistants. In *International Conference on Machine Learning (ICML)*.
- [14] Yang, K., Swope, A.M., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R. & Anandkumar, A. (2023) LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- [15] Zhang, H., Da, J., Lee, D., Robinson, V., Wu, C., Song, W., Zhao, T., Raja, P., Slack, D., Lyu, Q., Hendryx, S., Kaplan, R., Lunati, M. & Yue, S. (2024) A careful examination of large language model performance on grade school arithmetic. *arXiv:2405.00332*.
- [16] Zheng, K., Han, J.M. & Polu, S. (2022) miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*.

A Technical appendices and supplementary material

A. Outcome taxonomy in full. Table 1 in condensed form, plus the exact predicate each outcome is decided by and the order in which they are checked. Outcomes are assigned by the first matching predicate, in this order:

1. `PASS` — the final submission compiles cleanly (bare `lean <file>` from the repository root, exit status zero; §D below) *and* contains no hole token (`sorry`, `admit`, `Admitted`, `oops`, matched as whole words) *and* no axiom token (`axiom`, `Axiom`, `axiomatization`) *and* survives the statement-preservation guard below.
2. `trivial` — the record’s challenge and solution are byte-identical, so there is nothing to solve. Decided from the record alone, before the model is invoked. The ablaters refuse to emit such records, so this guards legacy data; it did not occur in the grid.
3. `malformed` — the challenge itself fails its pre-flight compilation, i.e. the problem was broken before the model saw it (an ablator or build-environment bug, not a solver failure).
4. `context_exceeded` — the provider rejected the prompt as exceeding the model’s context window. The model never saw the problem, so the attempt is excluded from the denominator.
5. `tampered` — the submission compiles hole-free, but a holed theorem was deleted or its statement weakened. For Lean the guard extracts each holed theorem’s statement from the challenge (declaration keyword through the `:=` proof delimiter), normalises whitespace, and requires that exact statement to appear in the submission; where the statement cannot be isolated (and for Coq/Isabelle, whose proof delimiters are less uniform) it falls back to requiring the declaration by name.

- 653 6. `gave_up` — the solver invoked its `give_up` tool, with a stated reason.
- 654 7. `turn_limit` — the request budget was exhausted with no hole-free compiling submission
- 655 on file.
- 656 8. `harness_err` — a transport-layer or provider failure (or a harness-side exception) ended
- 657 the attempt without an answer. Kept in the denominator as a non-pass (§3.6).
- 658 9. `fail` — none of the above: the solver finished, and its best submission does not compile
- 659 hole-free.

660 Holes are permitted *during* iteration: a submission containing `sorry` is compiled with holes allowed
661 and the solver is told whether the rest type-checks and how many holes remain, but such a submission
662 is never scored as solved. Axiom tokens are rejected at submission time rather than scored.

663 **B. Per-repository license table.** The full survey at each repository’s pinned revision, including the
664 five flagged entries discussed in §4.3: [‡] non-commercial-only grant, [†] no license file at the pinned
665 revision. Classification is of the license file text at the pinned revision itself, not the platform’s
666 default-branch metadata; the exact license-file URL at that revision for every repository is recorded
667 in `pipeline/licenses.tsv` in the artifact.

Table 9: Per-repository license survey at the pinned revision. Upstream is the GitHub owner; the repository name matches the corpus name except in the four rows showing a full `owner/name`.

Corpus name	Upstream	SPDX at pinned revision
adler-proofs	septract	BSD-3-Clause
aeneas	AeneasVerif	Apache-2.0
apc-optimizer	powdr-labs	MIT
ArkLib	Verified-zkEVM	Apache-2.0
btc-verified	ProofOfKeags	Apache-2.0
capless-lean	Linyxus	MIT
cedar-spec	cedar-policy	Apache-2.0
clean	Verified-zkEVM	MIT
Clear [‡]	NethermindEth	LicenseRef-Nethermind-NonCommercial
CompPoly	Verified-zkEVM	Apache-2.0
cslib	leanprover	Apache-2.0
curve25519-dalek-lean-verify	Beneficial-AI-Foundation	Apache-2.0
CvxLean	verified-optimization	Apache-2.0
dolev-yao	metareflection	MIT
evm-asm	Verified-zkEVM	MIT
FLoPS	rutgers-apl	MIT
formal-snarks-project	BoltonBailey	MIT
Foundation	FormalizedFormalLogic	Apache-2.0
FVIntmax	NethermindEth	Apache-2.0
hax-proof-libs-lean	cryspen/hax	Apache-2.0
hex-dev	kim-em	Apache-2.0
iris-lean	leanprover-community	Apache-2.0
jolt-tla	CharlesHoskinson	Apache-2.0
juvix-lean	anoma	MIT
lampe	reilabs	Apache-2.0
lean-formal-reasoning-program	andaraiwin	MIT
lean-mlir	opencompl	Apache-2.0
lean-yjs	iasakura	MIT
lean-zip	kim-em	Apache-2.0
lean4lean	digama0	Apache-2.0
leanda	konnov	MIT
Lentil	verse-lab	Apache-2.0
LeroyCompilerVerificationCourse	leanprover	LGPL-2.1
LNSym	leanprover	Apache-2.0
loom	verse-lab	Apache-2.0
nickelean	lexicone42	MIT
posix-parsing [†]	keeganperry7	none (no license file)
posix-submatching [†]	keeganperry7	none (no license file)
proven-zk	reilabs	Apache-2.0
ryu-lean4	lexicone42	MIT
SampCert	leanprover	Apache-2.0
shortest-decimal	lexicone42	MIT
SizzLean	etheorem/etheorem	LGPL-3.0
sparkle	Verilean	Apache-2.0
splean	verse-lab	Apache-2.0
starkware-formal-proofs [‡]	starkware-libs/formal-proofs	none (no license file)
symcrust	microsoft/SymCrypt	MIT
TensorLib	leanprover	Apache-2.0
TorchLean	lean-dojo	MIT

Corpus name	Upstream	SPDX at pinned revision
TTBFL	ionathanch	Zlib
VCV-io	Verified-zkEVM	Apache-2.0
veil	verse-lab	Apache-2.0
verified-compiler	marcusrossel	MIT
verified-consensus [†]	fradamt	none (no license file)
verity	lfglabs-dev	MIT
yul-compiler	powdr-labs	Apache-2.0
yul-semantics	powdr-labs	Apache-2.0

668 **C. Per-repository challenge counts.** Mined, malformed, unwinnable-ground-truth, and validated
669 counts per repository per holing strategy. Mined candidates are the same selections under both
670 strategies (§3.4); “unw.” is a mined candidate whose ground-truth solution failed to compile and was
671 therefore withheld; validated is what ships.

Table 10: Per-repository challenge counts by holing strategy.

Repository	Mined	Leaf holing			Whole-body holing		
		Malf.	Unw.	Valid.	Malf.	Unw.	Valid.
adler-proofs	43	0	35	8	0	35	8
aeneas	81	15	6	60	16	6	59
apc-optimizer	130	2	0	128	2	0	128
ArkLib	587	150	138	299	147	141	299
btc-verified	27	0	2	25	0	2	25
capless-lean	93	0	0	93	0	0	93
cedar-spec	1,107	74	0	1,033	69	0	1,038
clean	334	75	21	238	69	26	239
Clear	28	2	0	26	2	0	26
CompPoly	1,269	311	31	927	288	53	928
cslib	409	102	23	284	98	25	286
curve25519-dalek-lean-verify	227	47	12	168	47	12	168
CvxLean	29	4	0	25	4	0	25
dolev-yao	1	0	0	1	0	0	1
evm-asm	7,407	10	74	7,323	5	74	7,328
FLoPS	183	22	10	151	18	14	151
formal-snarks-project	12	1	0	11	1	0	11
Foundation	1,345	182	137	1,026	170	142	1,033
FVIntmax	49	2	0	47	2	0	47
hax-proof-libs-lean	60	14	0	46	0	0	60
hex-dev	3,403	328	70	3,005	339	75	2,989
iris-lean	379	44	22	313	41	22	316
jolt-tla	86	7	54	25	5	56	25
juvix-lean	19	2	0	17	2	0	17
lampe	324	83	9	232	92	14	218
lean-formal-reasoning-program	51	15	23	13	14	24	13
lean-mlir	237	72	76	89	72	76	89
lean-yjs	102	4	2	96	4	2	96
lean-zip	626	2	0	624	2	0	624
lean4lean	244	86	43	115	81	46	117
leanda	132	9	0	123	9	0	123
Lentil	39	4	1	34	4	1	34
LeroyCompilerVerificationCourse	18	2	1	15	0	1	17
LNSym	115	11	3	101	8	3	104
loom	39	11	0	28	11	0	28
nickelean	57	40	0	17	40	0	17
posix-parsing	13	0	0	13	0	0	13
posix-submatching	8	0	0	8	0	0	8
proven-zk	10	0	0	10	0	0	10
ryu-lean4	56	0	0	56	0	0	56
SampCert	177	10	0	167	2	0	175
shortest-decimal	129	0	0	129	0	0	129
SizzLean	38	27	3	8	27	3	8

Repository	Mined	Malf.	Unw.	Valid.	Malf.	Unw.	Valid.
sparkle	349	2	18	329	0	18	331
splean	185	28	88	69	28	88	69
starkware-formal-proofs	910	381	49	480	379	52	479
symcrust	367	11	34	322	9	34	324
TensorLib	2	0	0	2	0	0	2
TorchLean	740	27	13	700	26	13	701
TTBFL	172	24	13	135	23	13	136
VCV-io	1,469	640	57	772	613	83	773
veil	6	0	0	6	0	0	6
verified-compiler	6	0	0	6	0	0	6
verified-consensus	273	1	0	272	1	0	272
verity	1,766	462	42	1,262	464	42	1,260
yul-compiler	164	9	0	155	9	0	155
yul-semantics	25	0	0	25	0	0	25
Total	26,157	3,355	1,110	21,692	3,243	1,196	21,718

672 **D. Solver details.** The solver is a ReAct loop with five tools:

- 673 • `list_proof_files(query)` — list the repository’s own proof files (`.lean/.v/.thy`),
674 optionally filtered by substring.
- 675 • `search(pattern, context)` — regex search over the repository *and* its vendored li-
676 braries (e.g. Mathlib under `.lake`), returning matches with surrounding context lines.
- 677 • `read_file(path, start, end)` — read any of those files, including library files, by line
678 range.
- 679 • `submit_solution(solution)` — write the full file into the work copy, compile it, and re-
680 turn the result: for a hole-free submission, the compiler errors (or success); for a submission
681 still containing `sorry/admit`, whether the rest type-checks and how many holes remain
682 (feedback only, never scored); `axiom/axiomatization` are rejected outright.
- 683 • `give_up(reason)` — concede, recorded as the `gave_up` outcome.

684 Every tool call counts against the request budget, every tool result reports the remaining budget, and
685 the exploration tools are auto-disabled when few requests remain, leaving only `submit_solution`.
686 Each challenge runs in a fresh work copy of the source tree in which heavyweight build directories
687 are symlinked to the pristine checkout, so attempts cannot contaminate each other or the source.

688 The scoring compile for Lean is a bare `lean <file>` invocation from the nearest enclosing package
689 root with a 600 s timeout — never `lake env lean`, which resolves the workspace from the current
690 directory and, in a symlink-overlay work copy, follows the `.lake` symlink and writes into the shared
691 source tree (a failure mode that cost an earlier corpus build $\sim 3,700$ valid challenges before it was
692 found). `LEAN_PATH` is reconstructed by globbing `.lake/build/lib{,}/lean{}` of the package and
693 each of its vendored dependencies, and the FFI environment a C-extension package needs (`LEAN_CC`,
694 `LD_LIBRARY_PATH`, ...) comes from a single read-only `lake -no-build env printenv` snap-
695 shot taken once per pristine repository. Coq and Isabelle challenges use `coqc` against prebuilt `.vo`
696 dependencies and a session-aware `isabelle` build with prebuilt session images respectively, under
697 the same outcome taxonomy.

698 The system prompt, verbatim (`{assistant}` is the prover name and `{budget}` the request budget):

699 You are a proof engineer. The following `{assistant}` proof contains one or more holes
700 that are left to be proved. Your job: edit the file so it compiles properly, deriving
701 any helper lemmas as you need.

702
703 Rules:

- 704 - Your final (scored) file MUST compile and MUST NOT contain `'sorry'`, `'admit'`,
705 `'Admitted'`, `'oops'`, or any `'axiom'/'axiomatization'`. No cheating.
- 706 - You MAY submit a PARTIAL proof with `'sorry'/'admit'` placeholders while iterating: it is
707 compiled with holes allowed and you get feedback -- whether the rest type-checks and how
708 many holes remain -- so you can lock in the structure first, then fill each hole. A file

```

709   with holes is never scored as solved (only a fully hole-free, compiling file is), and
710   'axiom'/'axiomatization' are rejected outright.
711 - You may explore proof files (.thy/.lean/.v) to learn definitions, lemmas, and idioms:
712   'list_proof_files' lists the repo's own files; 'search' regex-searches the repo AND its
713   vendored libraries (e.g. mathlib under '.lake') and shows matches with surrounding
714   context; 'read_file' reads any of them (including library files like
715   '.lake/packages/mathlib/...'). Prefer 'search' to locate a relevant lemma, then
716   'read_file' it.
717 - You may NOT access the internet or git history. Work only from the files provided.
718 - Iterate: call 'submit_solution' with the full file; it compiles it and returns the
719   compiler errors (and, for a partial submission, the count of remaining holes). Fix and
720   resubmit until it compiles hole-free, or 'give_up' with a reason.
721 - BUDGET: you have at most {budget} model requests -- every 'read_file',
722   'list_proof_files', 'search', and 'submit_solution' call counts, and you are
723   HARD-CUT-OFF when the budget is exhausted (no final turn). Every tool result reports
724   your remaining budget ('turns_left' / '[budget: ...]'); WATCH IT. Submit a COMPLETE
725   first attempt EARLY (within your first few turns) -- a full file you then refine against
726   compile errors -- rather than exploring until you run out. Don't over-explore;
727   'read_file'/'search' are AUTO-DISABLED once few requests remain, leaving you only
728   'submit_solution'. Only a fully hole-free submission is scored, so always get at least
729   one complete attempt submitted.

```

730 **E. Broader impacts.** The corpus is proof code redistributed from public repositories under the
731 licenses surveyed in §4.3; it contains no personal data and no dual-use capability of its own. The
732 intended positive impact is to make machine-assisted proof engineering measurable on realistic
733 work, which is a prerequisite for lowering the cost of formally verifying safety- and security-critical
734 software. Two risks are worth naming. First, a benchmark that is optimised against can be gamed at
735 the level of the *statement* rather than the proof; §5.3 measures exactly this, and the guard of §3.7 is a
736 partial mitigation that we recommend any verified-reward environment adopt. Second, the corpus is
737 drawn from projects whose authors did not release their work as an evaluation set; we mitigate by
738 pinning and citing every source repository at its exact revision, surveying licenses at that revision, and
739 flagging rather than silently including repositories whose terms do not clearly permit redistribution
740 (§4.3).

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Section 1 state exactly five claims, each with a forward reference to the section that discharges it, and Section 6 states the scope conditions on each. Where a stated hypothesis was not borne out we say so in the same place we stated it: the premise that whole-body holing is harder is reported as a null in Section 5.4, and the prediction that tamper rate rises with turn budget is reported as unconfirmed in Section 5.3. Claims whose supporting runs have not yet completed are marked in the text rather than asserted.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 6 is a dedicated Limitations section covering the single-prover scope, the budget-dependence of every reported pass rate, the built-checkout requirement for scoring, corpus concentration, context-window exclusions, and unresolved licensing on part of the corpus.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theoretical results; the contribution is a dataset, a construction method, and empirical measurements.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3 gives the full construction (corollary closure, deletion, holing strategy, corollary-anchored slicing, deterministic seed, and the two-sided compile-validation criteria), Section 4 gives the pinned (repository, revision, toolchain) provenance for all 57 sources, and Section 5.1 plus Appendix D give the solver loop, tools and scoring invocation. Section 3.5 additionally documents the build-environment failure modes that silently invalidate this kind of pipeline.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The mined corpus, the ablation tooling, the evaluation harness and the pipeline scripts are released, with an anonymized artifact link given in Section 4. Section 6 (L3) is explicit that scoring additionally requires a built checkout of the source repository and that prebuilt closures are published only for a demo subset.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5.1 specifies the solver loop, the tool set, the sample-construction rule (up to two matched `challenge_ids` per repository at seed 42, giving 113 pairs), the three model identifiers, the 50-turn budget applied uniformly, and the exclusion rules that take the 113 pairs to 103 scorable. Section 3.6 and Table 1 give the scoring rule, and Appendix D gives the solver prompt, tool schemas and compile invocation. The one experimental variable we have not swept is the turn budget itself, which Section 6 (L2) states as a limitation rather than leaving implicit.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.

- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Every rate in Table 3 carries a bootstrap 95% interval, resampling repositories for the macro-average and problems for the micro-average, and the resampled unit is stated in Section 5.1. The paired holding-strategy comparison uses an exact (binomial) McNemar test on the discordant pairs of a matched `challenge_id` set, with the discordant counts reported alongside every p -value (Section 5.4). Section 6 (L8) states what these intervals do *not* capture: a single seed and a single run per cell, so solver run-to-run variance is not represented. The temporal-holdout cells in Table 6 are reported without intervals because the pre-cutoff side has 10–19 problems, and the text says so rather than presenting an underpowered comparison as a result.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Table 5 reports, per model, the average number of solver requests, the input and output token counts, and the aggregate solver wall-clock time for the six runs of the main grid: 407.6M input tokens, 8.9M output tokens and 117.5 hours in total over 678 solve attempts. Inference runs against hosted APIs; all proof compilation — mining, two-sided validation, and the per-submission scoring compile inside the solver loop — runs CPU-only on a single workstation, and no accelerators are used anywhere in the pipeline. Preliminary and failed runs preceding the reported grid are not included in those totals.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work uses only publicly available source repositories, redistributes them under a per-repository license survey conducted at each pinned revision (Section 4.3), involves no human subjects and no personal data, and flags rather than silently includes sources whose terms do not clearly permit redistribution.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Appendix E discusses both the intended benefit (making machine-assisted proof engineering measurable on realistic work, as a prerequisite for cheaper formal verification of critical software) and the risks (statement-level reward hacking, which Section 5.3 measures directly, and redistribution of work whose authors did not release it as an evaluation set).

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

1009 Answer: [N/A]

1010 Justification: The release is proof source code from public repositories and a scoring harness;
 1011 it contains no personal data, no generative model weights, and no capability that carries a
 1012 high risk of misuse. The one release-specific hazard we identified — benchmark gaming by
 1013 weakening the target statement — is addressed by the tamper guard described in Section 3.7.

1014 Guidelines:

- 1015 • The answer [N/A] means that the paper poses no such risks.
- 1016 • Released models that have a high risk for misuse or dual-use should be released with
 1017 necessary safeguards to allow for controlled use of the model, for example by requiring
 1018 that users adhere to usage guidelines or restrictions to access the model or implementing
 1019 safety filters.
- 1020 • Datasets that have been scraped from the Internet could pose safety risks. The authors
 1021 should describe how they avoided releasing unsafe images.
- 1022 • We recognize that providing effective safeguards is challenging, and many papers do
 1023 not require this, but we encourage authors to take this into account and make a best
 1024 faith effort.

1025 **12. Licenses for existing assets**

1026 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
 1027 the paper, properly credited and are the license and terms of use explicitly mentioned and
 1028 properly respected?

1029 Answer: [Yes]

1030 Justification: All 57 source repositories are credited with their URL and exact revision, and
 1031 Section 4.3 plus Appendix B give the per-repository SPDX classification derived from the
 1032 license text at that pinned revision (not at default-branch HEAD, which we show is the
 1033 wrong thing to trust). Five repositories whose terms are restrictive or absent are flagged
 1034 rather than quietly redistributed. **[TODO: record the final keep/drop decision on those**
 1035 **five]**

1036 Guidelines:

- 1037 • The answer [N/A] means that the paper does not use existing assets.
- 1038 • The authors should cite the original paper that produced the code package or dataset.
- 1039 • The authors should state which version of the asset is used and, if possible, include a
 1040 URL.
- 1041 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 1042 • For scraped data from a particular source (e.g., website), the copyright and terms of
 1043 service of that source should be provided.
- 1044 • If assets are released, the license, copyright information, and terms of use in the
 1045 package should be provided. For popular datasets, `paperswithcode.com/datasets`
 1046 has curated licenses for some datasets. Their licensing guide can help determine the
 1047 license of a dataset.
- 1048 • For existing datasets that are re-packaged, both the original license and the license of
 1049 the derived asset (if it has changed) should be provided.
- 1050 • If this information is not available online, the authors are encouraged to reach out to
 1051 the asset’s creators.

1052 **13. New assets**

1053 Question: Are new assets introduced in the paper well documented and is the documentation
 1054 provided alongside the assets?

1055 Answer: [Yes]

1056 Justification: The released corpus ships per-repository manifests recording source repository,
 1057 revision, toolchain, mined/validated/dropped counts and blob checksums, together with a
 1058 dataset card documenting the construction, the outcome taxonomy, the aggregate license
 1059 position and the known limitations.

1060 Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work involves no crowdsourcing and no human subjects; all data is mined automatically from public source repositories.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work involves no human subjects, so no IRB or equivalent review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

1111 Justification: Large language models are a core, declared component: they are the subject of
1112 evaluation, and the solver of Section 5.1 is an LLM-driven ReAct agent whose tool loop and
1113 compile-feedback design are part of the method. Section 5.5 additionally treats the models'
1114 training data as an object of measurement.

1115 Guidelines:

- 1116 • The answer [N/A] means that the core method development in this research does not
1117 involve LLMs as any important, original, or non-standard components.
- 1118 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1119 be described.